

Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Michael Shell, *Member, IEEE*, John Doe, *Fellow, OSA*, and Jane Doe, *Life Fellow, IEEE*

Abstract—The abstract goes here.

Index Terms—IEEE, IEEEtran, journal, LATEX, paper, template.

I. INTRODUCTION

THIS demo file is intended to serve as a “starter file” for IEEE journal papers produced under LATEX using IEEEtran.cls version 1.8b and later. I wish you the best of success.

mds

August 26, 2015

A. Subsection Heading Here

Subsection text here.

II. RELATED WORK

III. METHOD

We propose **V-LGP** (Vision-Language Geometric Programming), a hierarchical closed-loop framework that bridges high-level semantic reasoning with low-level geometric motion planning. The system operates through a “Perception-Reasoning-Action-Verification” pipeline, designed to solve long-horizon manipulation tasks by decomposing complex assembly goals into locally optimal, solvable sub-problems.

A. Phase 0: Spatio-Geometric Grounding

The pipeline is initialized with a top-down orthographic camera view and an anonymous simulation kinematic graph (.g file). To ensure Sim-to-Real robustness against lighting fluctuations, shadows, and reflections, V-LGP eschews brittle color-hashing in favor of a **Geometry-Distance Binding** protocol.

The **VLM Matcher** identifies object candidates based on their geometric primitives and sorts them according to their **Radial Euclidean Distance** from the robot base. By prompting the model to label objects based on this deterministic spatial ordering (e.g., `obj_near_01` assigned to the closest cylinder), the system autonomously binds raw visual features to semantic simulation IDs. A deterministic Python script then injects these human-readable semantic tags back into the underlying kinematic graph (`scene_named.g`).

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see <http://www.michaelshell.org/contact.html>).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

B. Phase 1: Zero-Shot Topology Extraction

To parse the *Visual Assembly Manual*, the **VLM Analyst** leverages the zero-shot reasoning capabilities of state-of-the-art models to extract a semantic **Layered Precedence Graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$.

Unlike traditional methods requiring heavy In-Context Learning (ICL), our approach utilizes the model’s inherent spatial common sense:

- 1) **Direct Visual Reasoning:** The model analyzes the live test scene and the target assembly configuration directly, identifying structural dependencies without prior visual examples.
- 2) **Deterministic Sandboxing:** The VLM outputs a hierarchical JSON of assembly nodes. A Python inspector rigorously validates this output against a strictly defined whitelist of 2D-native slots and objects, preventing the propagation of invalid frame IDs to the solver.

C. Phase 2: Dynamic Rule Allocation & Batch Compilation

This phase introduces the core computational breakthrough of V-LGP: **Dynamic Rule Allocation**. Traditional LGP solvers encounter an exponential branching factor b as the number of available predicates P increases, leading to a state-space complexity of $\mathcal{O}(b^P)$. V-LGP prunes this search tree by isolating only the predicates required for the current batch.

The **VLM Strategist** maps generic requests to the inventory and compiles minimal, specialized .lgp and .fol files. For a batch involving cylinder placement, the compiler exclusively injects specific rules such as `pick_cylinder` and `place_straightOn`. We quantify the complexity reduction as:

$$\text{Complexity}_{\text{V-LGP}} = \sum_{i=1}^N \text{solve}(\text{Node}_i, P_{\text{batch}}) \approx N \cdot \mathcal{O}(1) \quad (1)$$

where $P_{\text{batch}} \ll P$. This mechanism achieves a constant state-space footprint per node, reducing KOMO execution time from over ten minutes to a deterministic two-minute window.

D. Phase 3: Constrained Geometric Execution

The geometric execution phase maps symbolic logic to continuous trajectory optimization. We augment the native C++ LGP infrastructure to bridge the Sim-to-Real gap in complex, multi-support assemblies.

1) *Topological Decoupling via Virtual Anchors*: To support multi-support scenarios (e.g., a base resting on multiple cylinders), we introduce **Virtual Anchors** (A_v). Standard LGP formulations fail when multi-parenting induces a closed kinematic loop, disrupting Jacobian computations. A_v breaks the loop via a 6-DOF free joint (`JT_free`) parented to the world frame. The solver optimizes the state x_t by forcing the anchor to track the geometric centroid of support frames S_i :

$$\min_{x_t} \left(\lambda_1 \sum_{t'=t-1}^t \|\phi_{\text{pose}}(A_v, x_{t'}) - \mathbf{p}_{\text{target}}\|^2 + \lambda_2 \|\phi_{\text{poseDiff}}(A_v, O, x_t)\|^2 \right) \quad (2)$$

2) *Trajectory Shaping via Geometric Funnels*: To prevent lateral collisions during insertion, we enforce **Pre-Action Funnels**. For an insertion at time t , we inject relative positional constraints at waypoints $t - 0.5$ and $t - 0.1$. This forces the object O to align precisely above the target anchor A_v with a vertical standoff distance $d_z = 0.05\text{m}$:

$$\Phi_{\text{funnel}}(x_{t-\delta}) = \left\| \phi_{\text{posRel}}(O, A_v, x_{t-\delta}) - \begin{bmatrix} 0 & 0 & d_z \end{bmatrix}^T \right\|^2 = 0 \quad (3)$$

This mathematically constructs a “funnel” that forces strictly vertical approach and retract paths, mimicking human-like precision.

3) *Temporal Collision Relaxation*: We implement a phased collision protocol. During aerial transit ($t \in [t - 0.95, t - 0.8]$), a strict inequality constraint is enforced using a safety margin $d_{\text{safe}} = 0.01\text{m}$. During the final placement phase ($t - 0.1$ to t), this margin is hierarchically relaxed to allow physical contact, preventing solver deadlocks at the moment of surface integration.

E. Phase 4: State Verification & Structural Checkpoints

Upon completion of a phase, the **VLM Judicator** captures an updated image to verify the scene against the expected JSON state. Currently, this serves as a **Structural Checkpoint** to ensure safety; if the visual state deviates from the plan, the system triggers a `Safe Halt`. While this acts as a monitored open-loop mechanism, it establishes the foundation for future work in automated re-planning and impedance-based error recovery.

IV. EXPERIMENTAL RESULTS

1) *Subsubsection Heading Here*: Subsubsection text here.

V. CONCLUSION

The conclusion goes here.

APPENDIX A

PROOF OF THE FIRST ZONKLAR EQUATION

Appendix one text goes here.

APPENDIX B

Appendix two text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.
- [2] Placeholder Author, “Title of the Fabrica Reference,” Journal/Conference, Year.



Michael Shell Biography text here.

John Doe Biography text here.

Jane Doe Biography text here.